

A Heuristic Load-Balancing Algorithm for Multi-Mesh Simulations on GPUs

Leo Sitaraman,
Gr. 11, Homestead High School, Cupertino CA

Outline

- Introduction
- Motivation
- Algorithm description
- Canonical tests
- Application results
- Conclusions & Future Work

Introduction

- Computational Fluid Dynamics (CFD): used to simulate how objects interact with air flow
- Multiple meshes are often used to represent different components of a complex vehicle
- They require a lot of computing power → run on high-performance computing (HPC) clusters
- HPC clusters use many processors working together to solve the problem faster
- Traditionally, simulations are split across thousands of CPUs

Complex vehicle example

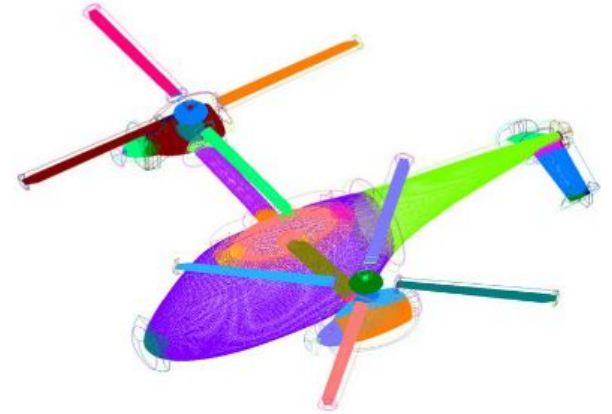


Figure from Ventura Diaz, P., and Yoon, S.,
“High-fidelity computational aerodynamics
of multi-rotor unmanned aerial vehicles,”
2018 AIAA Aerospace Sciences Meeting,
2018, p. 1266.

Motivation

- CFD codes increasingly using GPUs instead of CPUs
- 1 GPU matches the performance of 1000 CPU cores¹
- In the past: Many more CPUs than meshes
- Today: The same or fewer GPUs than meshes
- Current mesh distribution algorithms are optimized for cases with many more processors than meshes
 - These algorithms do not consider arrangements where multiple meshes are assigned to each processor

CPU



<https://www.extremetech.com/computing/>

GPU



<https://www.corsair.com/us/en/explorer/diy-builder/blogs/>

¹ Krakos, J., Gomes, P., Ho, J., Mara, M., Sáez, G., Loseille, A., Economon, T., and Alonso, J. J., "GPU-based and Adaptive Solution Technology for the 5th AIAA High Lift Prediction Workshop," AIAA SCITECH 2025 Forum, 2025, p. 0496.

Literature Survey

- Partitioning on CPUs were achieved using pro-rated distribution
 - Number of processors assigned is proportional to size of the mesh
 - Each processors typically has only one partition corresponding to one of the meshes
- Optimization of this algorithm was performed by Shrerer and Garman^{1,2}
 - Start with proportional partitioning
 - Perturb number of processors for each mesh in a small range
 - Recursive algorithm to search the space and identify an optimum
- These algorithms are unsuitable on GPUs because they assume
 1. Number of processors are much larger than number of meshes
 2. A single mesh partition per processor

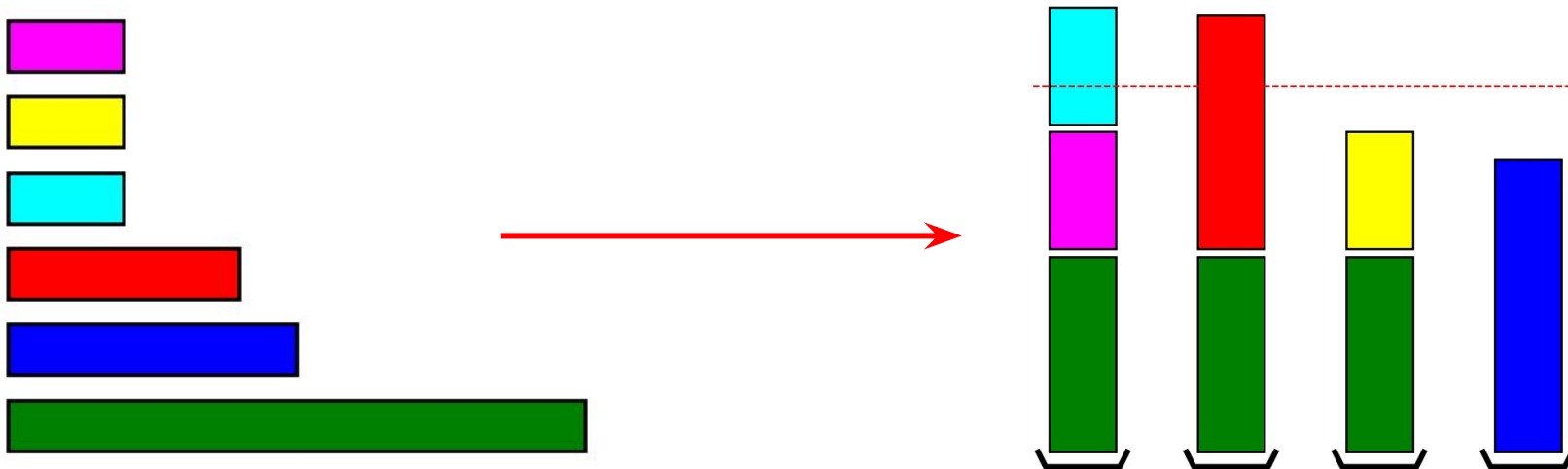
¹Sherer, S. E., and Garmann, D. J., "Automatic Partitioning of Structured Overset Grids for Use with High-Order Algorithms," 2018 AIAA Aerospace Sciences Meeting, 2018, p. 1405.

²Sherer, S. E., and Garmann, D. J., "Further Development and Performance Assessment of Automatic Partitioning Algorithms for Structured Overset Grids," AIAA Scitech 2019 Forum, 2019, p. 1987.

The goal:

Distribute a set of meshes of varying sizes across a set number of processing elements with the goal of:

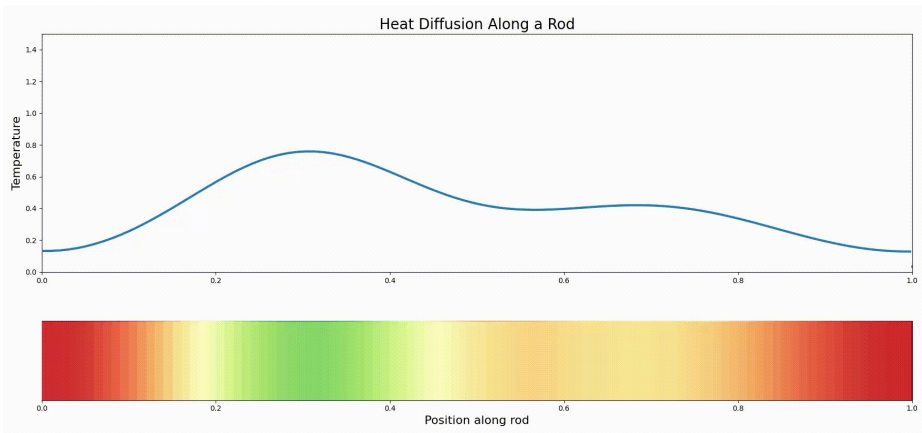
- a) Minimizing the maximum mesh load (combined mesh size) for a processor
- b) Minimizing the total number of mesh splits



Why minimize splits?

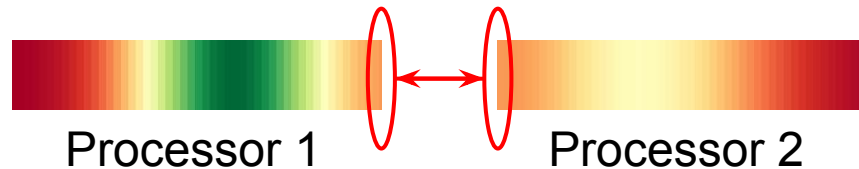
Consider the following 1D scenario:

A rod is heated to various temperatures along its length. We want to simulate how the heat at each point changes with time:



To simulate this situation, each segment of the bar must use its neighbor's temperature and its own temperature to determine how to change in a timestep.

At the edges where the rod is split, the states of these neighbors must be communicated between the processors, which takes non-negligible time.



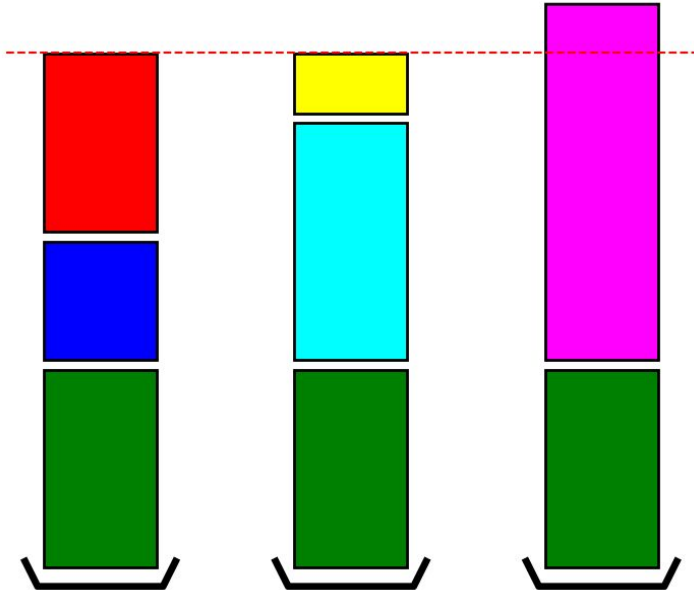
Why minimize splits?

This is also the case with 2D or 3D meshes: Along edges or planes where they are split, an additional overlapping layer of cells must be added and communicate data with corresponding edges on other pieces. This communication typically adds an amount of time proportional to the surface area of a piece.

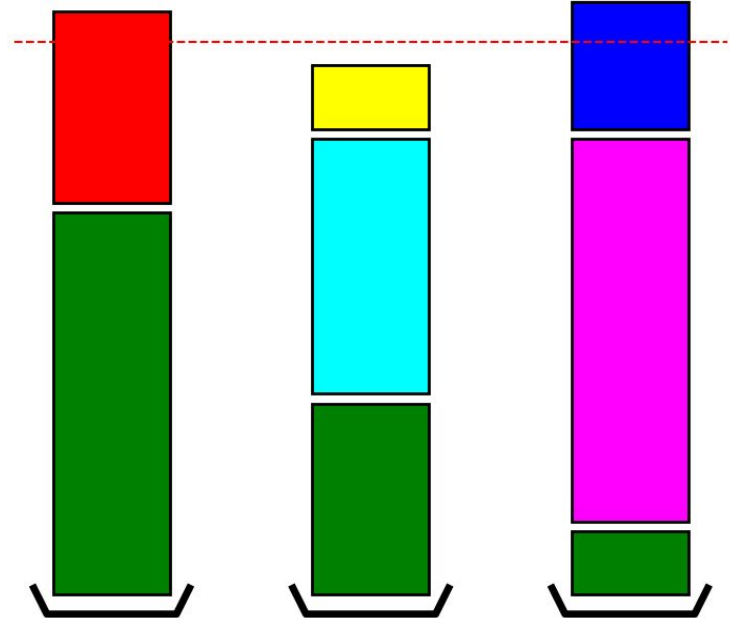
Constraints:

- 1) All divided meshes must be divided into equal parts

OK ✓



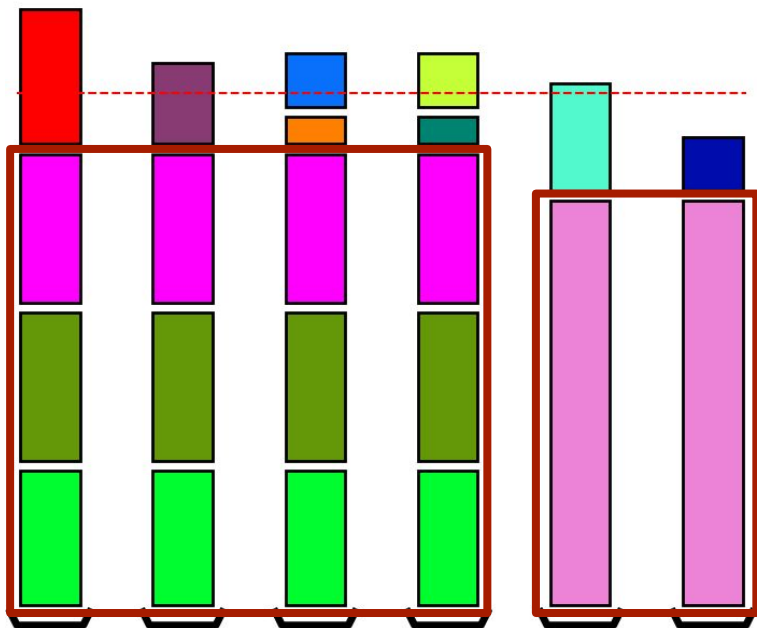
NOT OK ✗



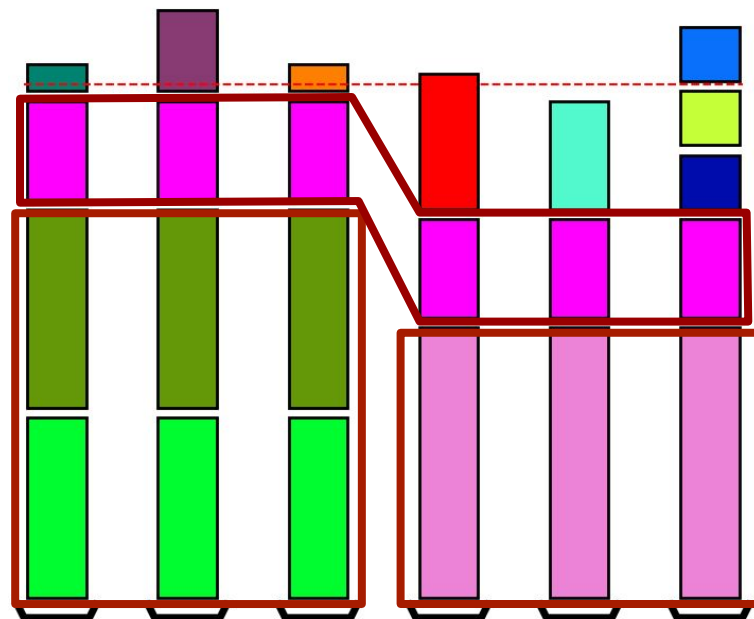
Constraints:

- 2) Groups of divided meshes can't overlap

OK ✓



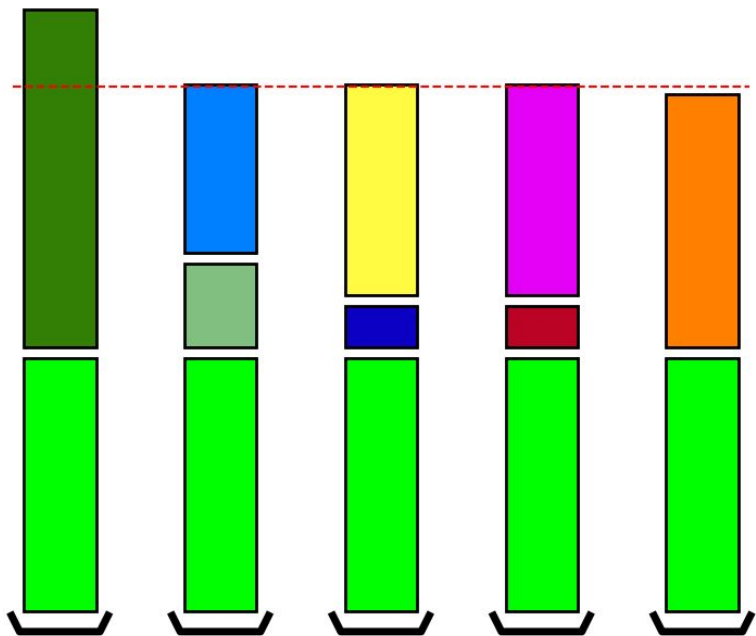
NOT OK ✗



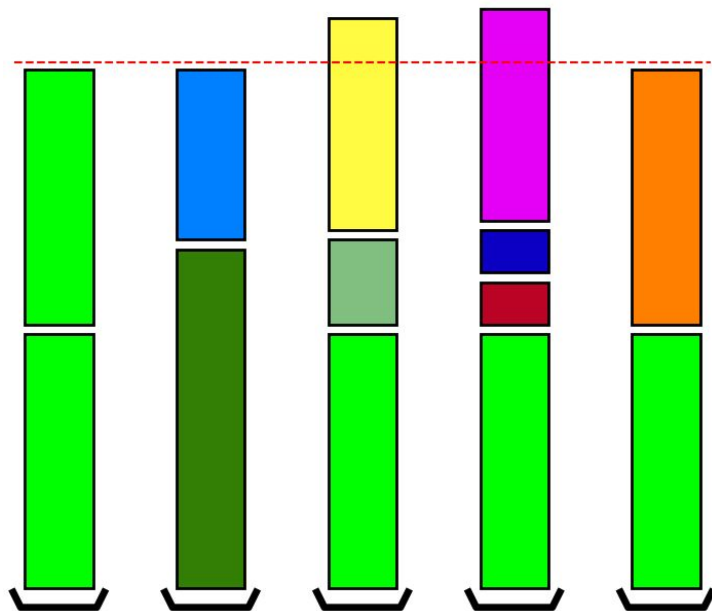
Constraints:

- 3) Parts of the same mesh can't be in the same processor

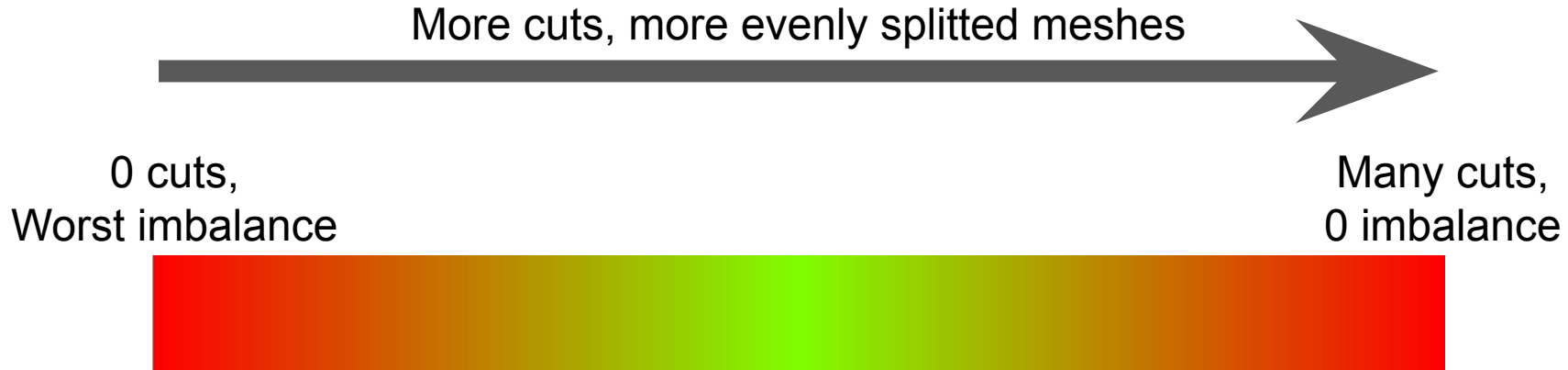
OK ✓



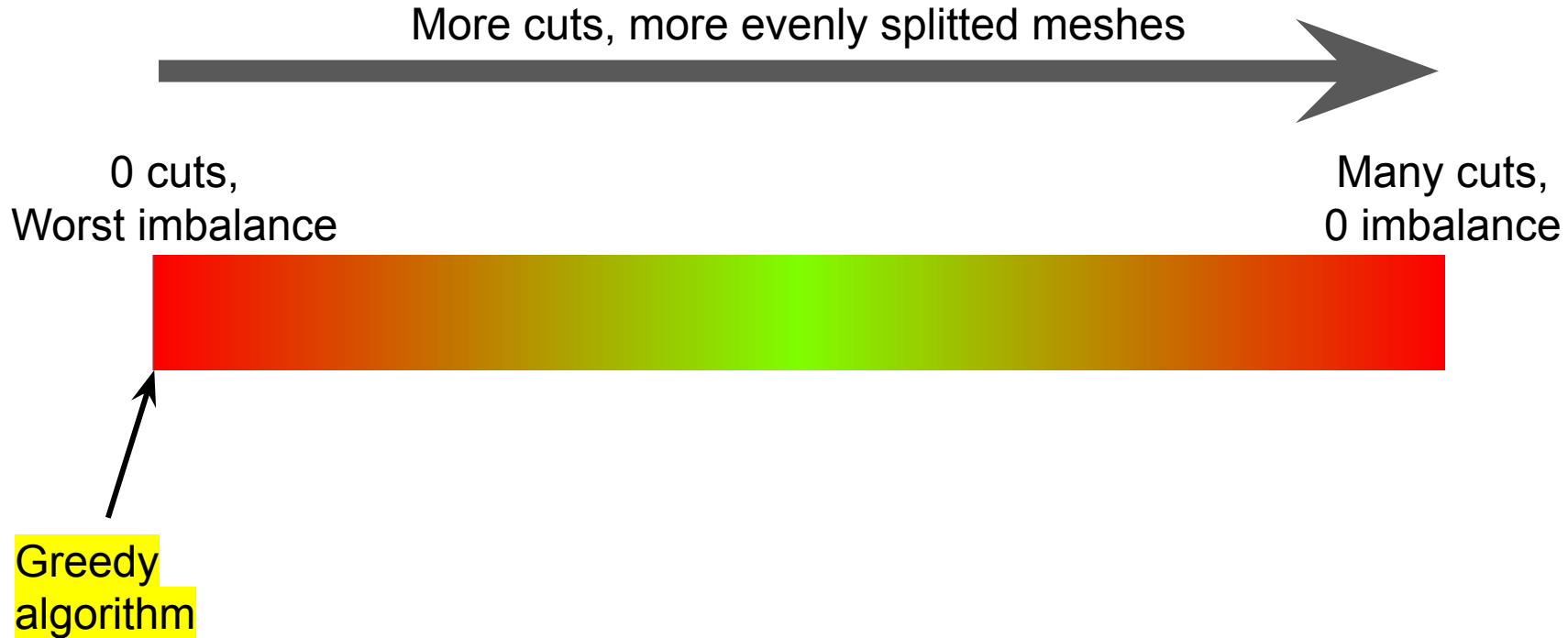
NOT OK ✗



Optimizing max load and number of cuts:



Optimizing max load and number of cuts:



Greedy Algorithm:

- 1) Sort meshes in descending order
- 2) Process meshes in order from largest to smallest, placing each mesh in the least-loaded processor

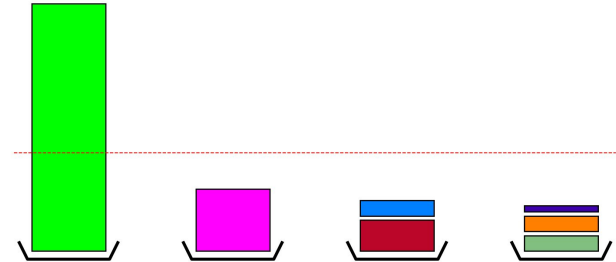


Pros:

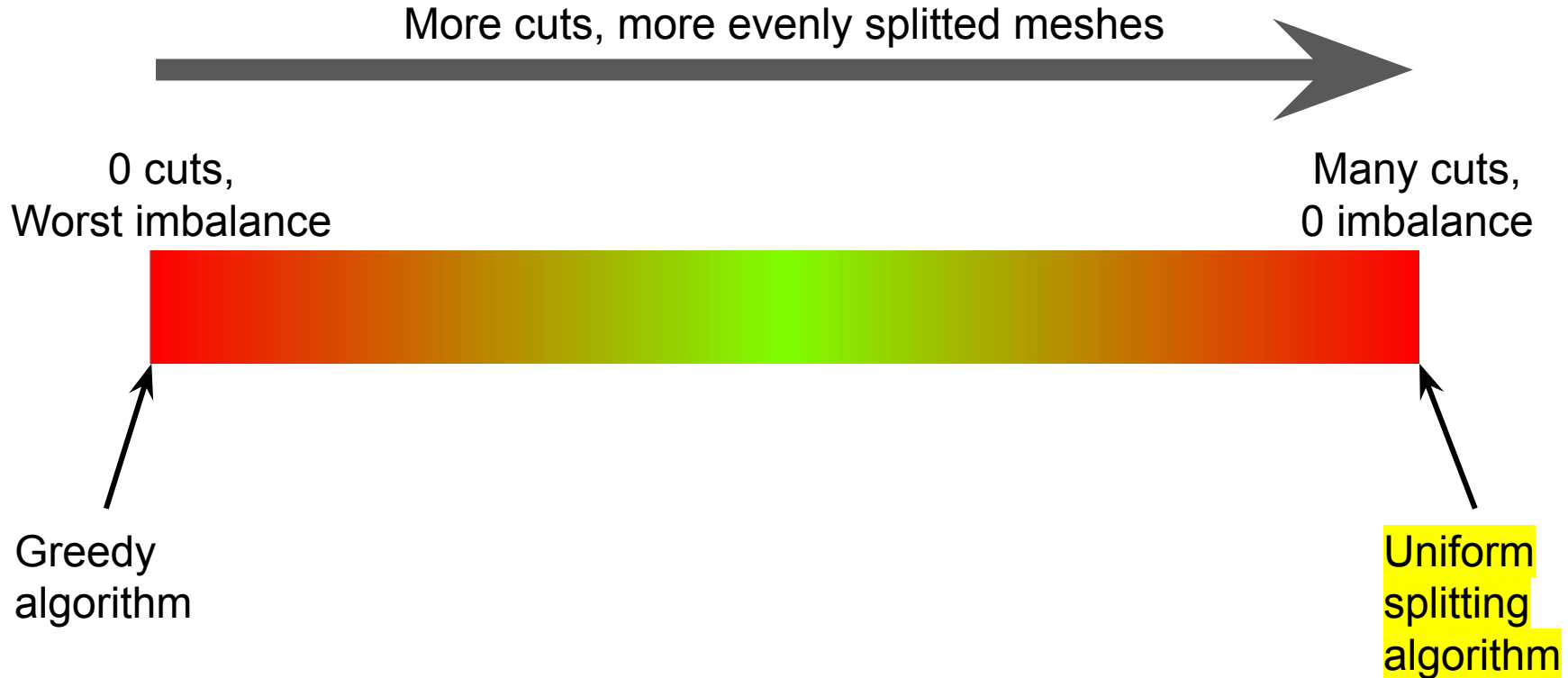
- Does not split any mesh, so no communication time is required

Cons:

- For certain mesh size combinations, results produced are very unbalanced:

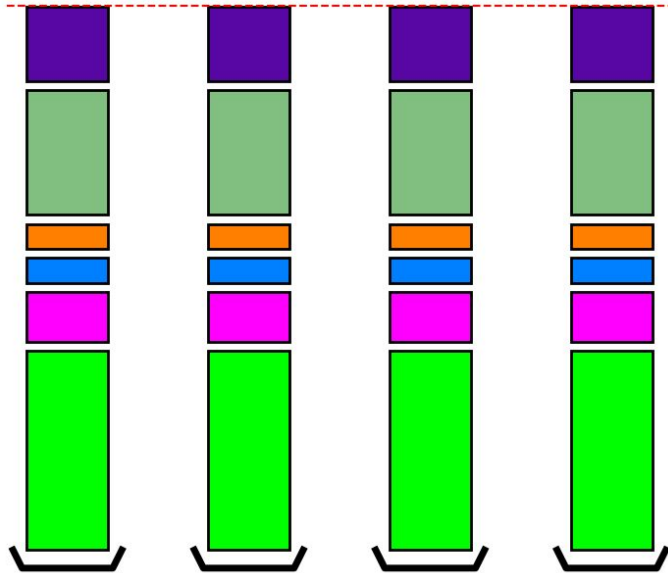


Optimizing max load and number of cuts:



Uniform Splitting Algorithm

- 1) For N processors, divide each mesh into N parts
- 2) Distribute 1 part of each mesh to each processor



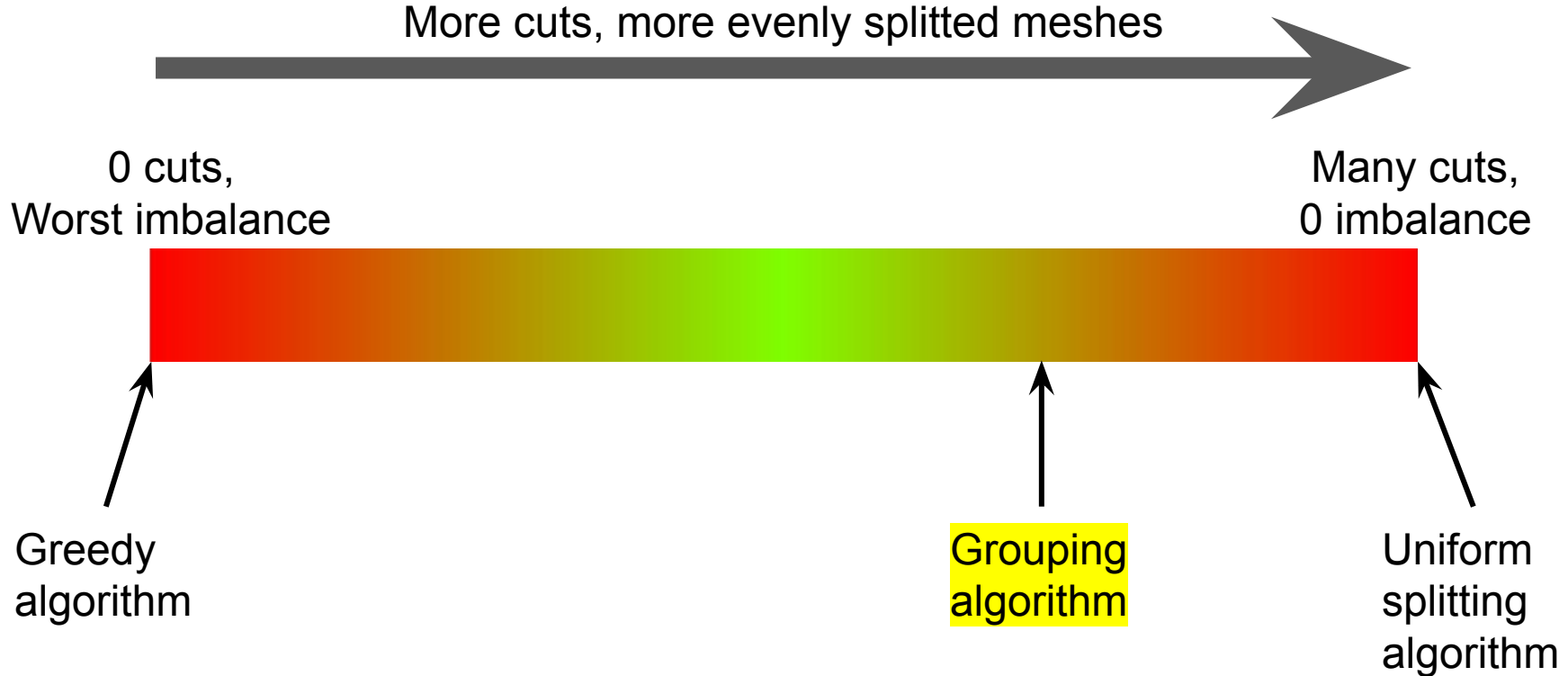
Pros:

- Perfectly balanced distribution, as all meshes are evenly split

Cons:

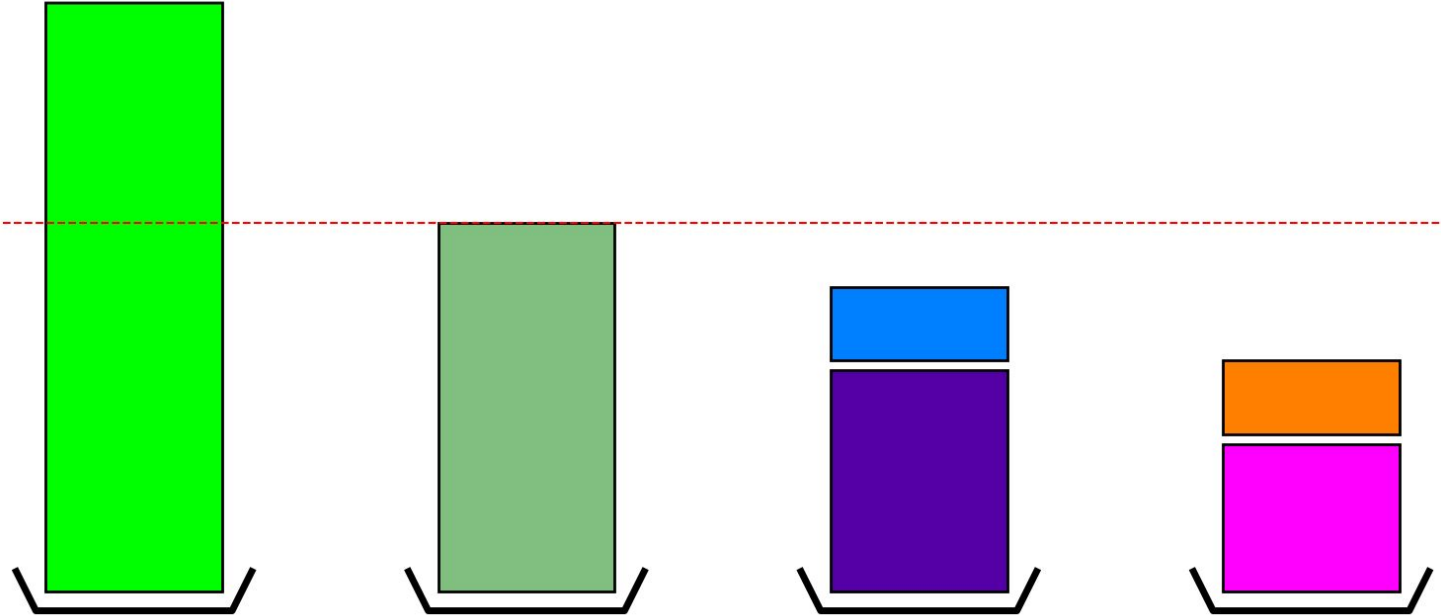
- Requires a large number of mesh splits, causing significant communication time and slowing down computations
- Splits all small meshes into every processor, leading to very small pieces of meshes which are inefficient for computation

Optimizing max load and number of cuts:



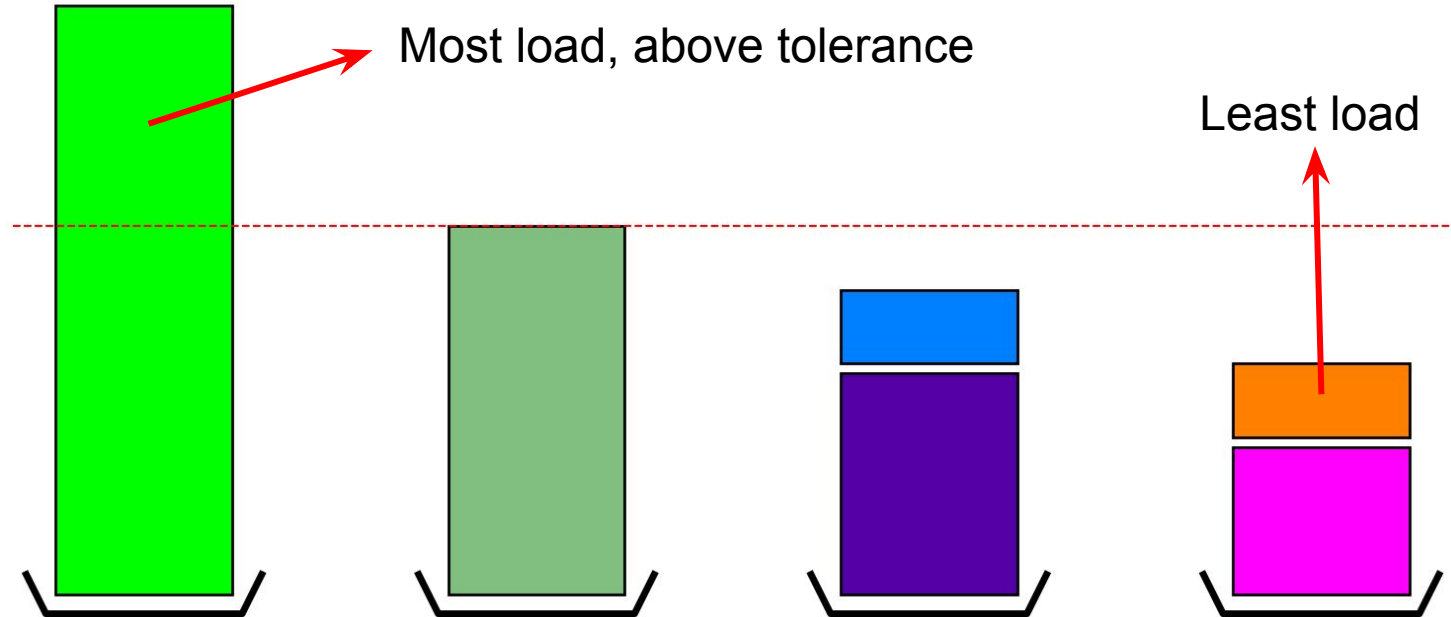
Grouping Algorithm

Step 1: Perform a greedy distribution for the meshes:



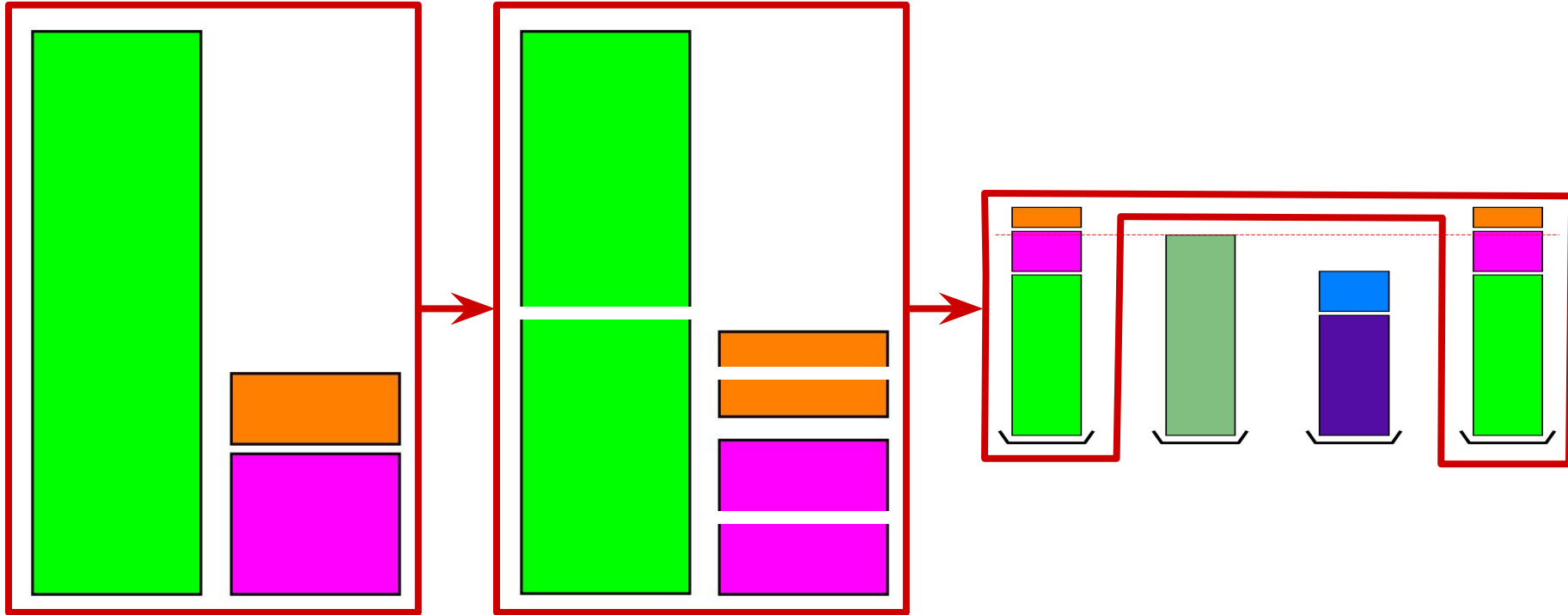
Grouping Algorithm

Step 2: Identify the processor with the most load. If it is above a tolerance from the ideal load (red dashed line), then combine it with the least-loaded processor.



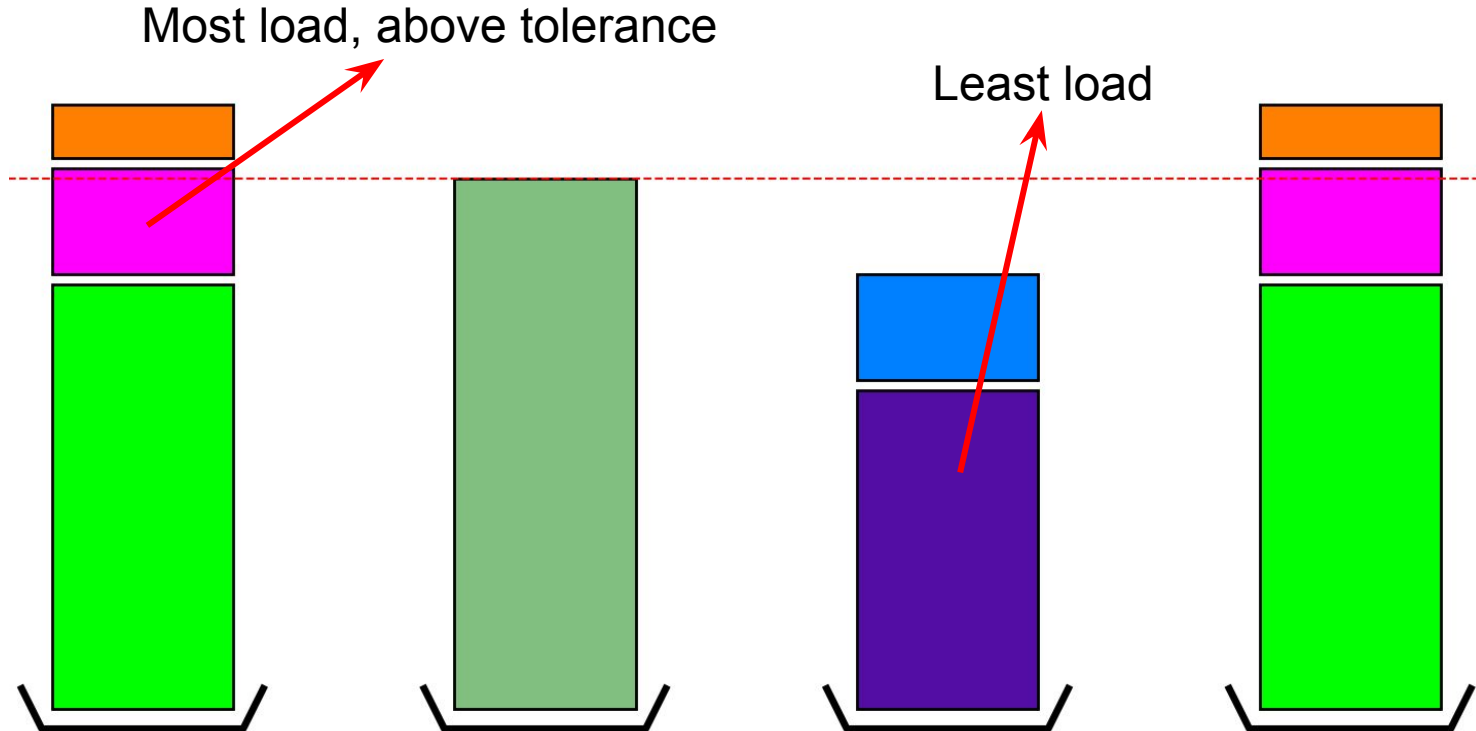
Grouping Algorithm

When two sets of meshes are combined, each mesh in the combined set is split apart and each processor receives one piece of each mesh.



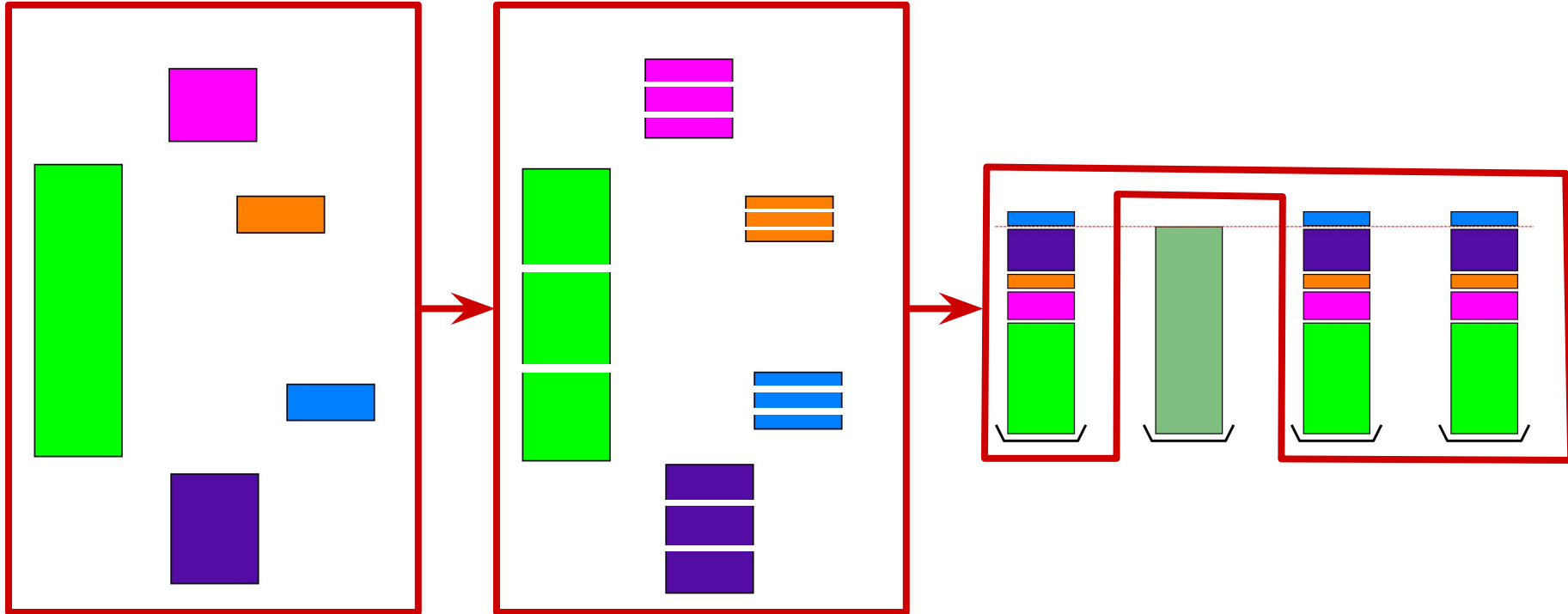
Grouping Algorithm

The process is repeated until all processors are below tolerance



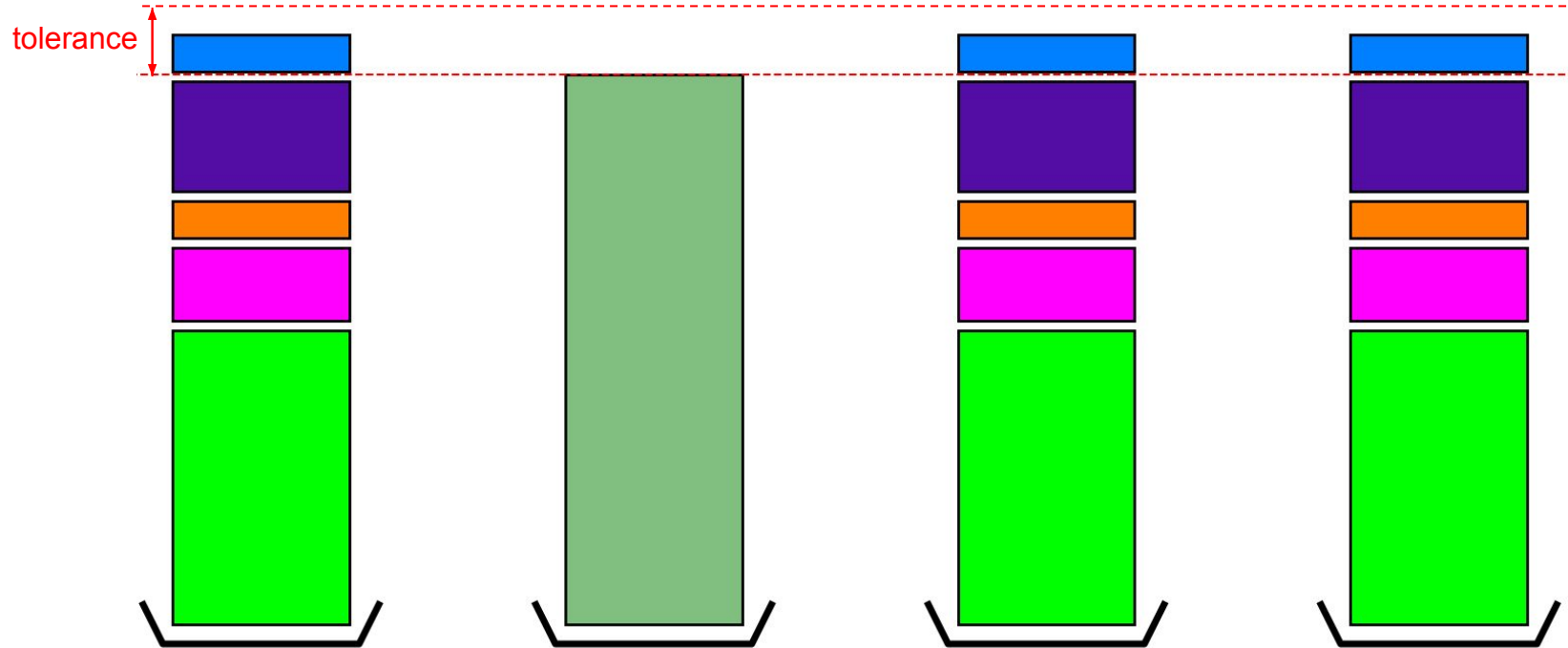
Grouping Algorithm

When an existing group gets combined once more, the number of divisions increases (in this case each mesh in the group is now divided into 3 instead of 2)

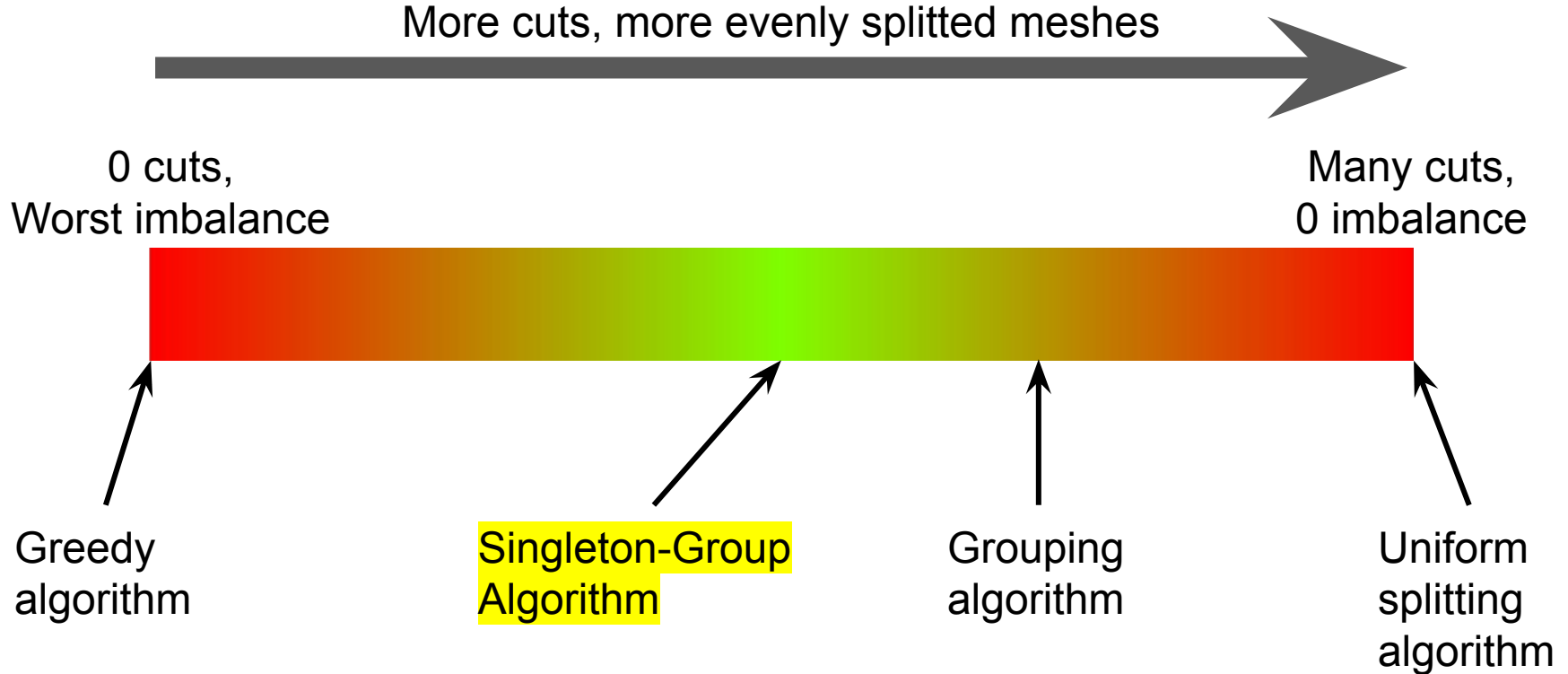


Grouping Algorithm

After 2 iterations, all processors are now within the tolerance



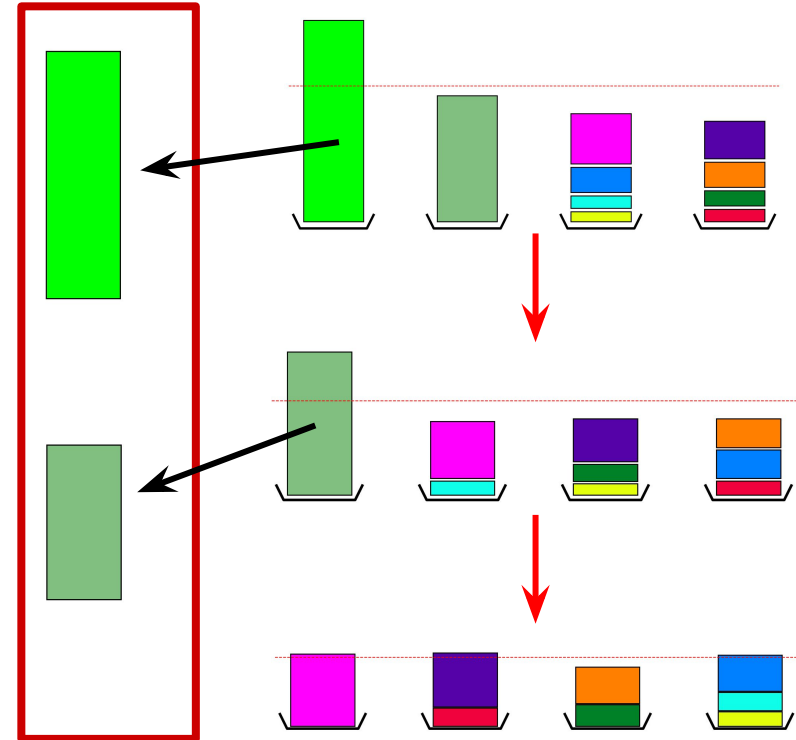
Optimizing max load and number of cuts:



Singleton-Group Algorithm

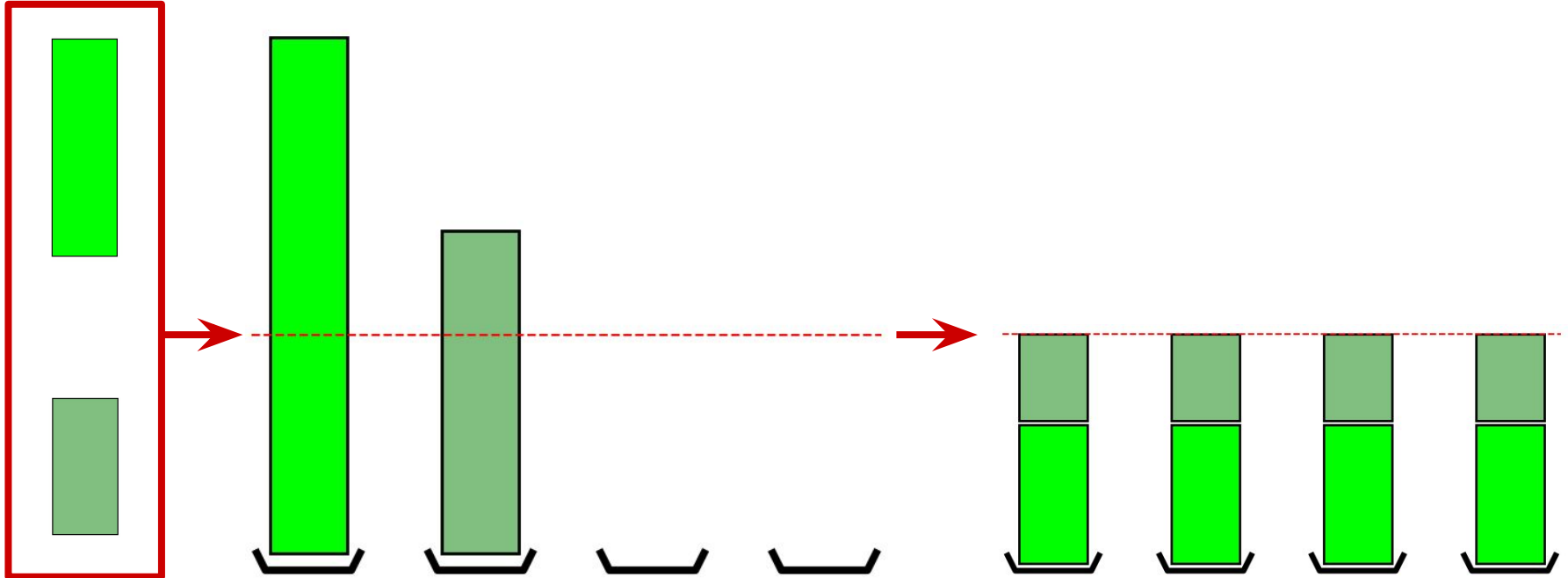
Phase 1: Determine a subset of the original meshes that can be distributed nicely with greedy algorithm

- 1) Perform a greedy distribution on all meshes.
- 2) Identify the worst loaded processor. If it is above the tolerance, remove the largest mesh in that processor from the current set and save it for later.
- 3) Perform a greedy distribution on the smaller set of meshes. Repeat steps 2 and 3 until all processors are within the tolerance.



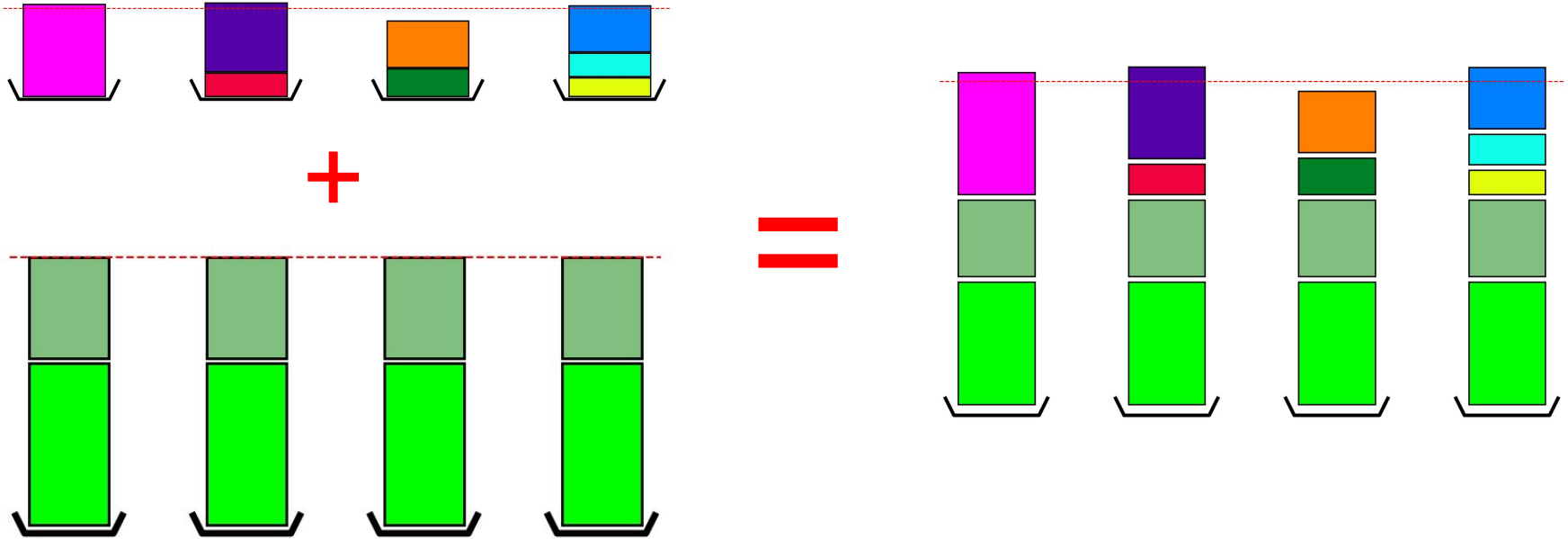
Singleton-Group Algorithm

Phase 2: Apply the grouping algorithm to the removed meshes from phase 1

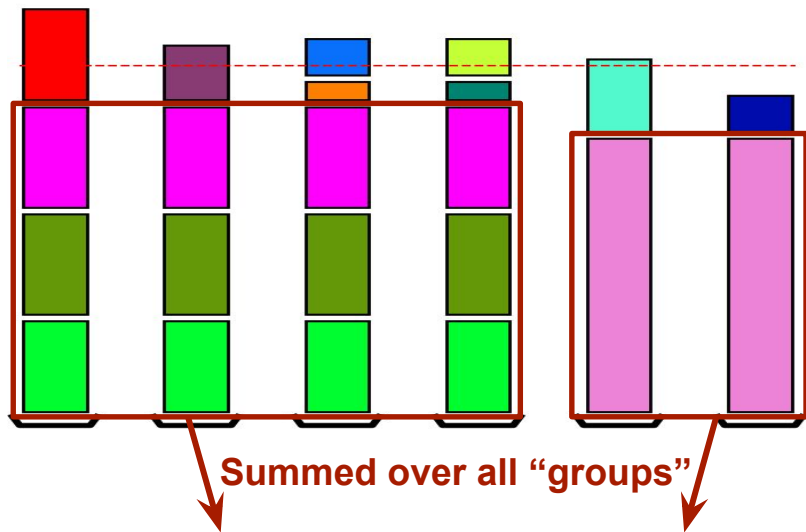


Singleton-Group Algorithm

Phase 3: Combine grouping distribution with greedy



Parallel efficiency



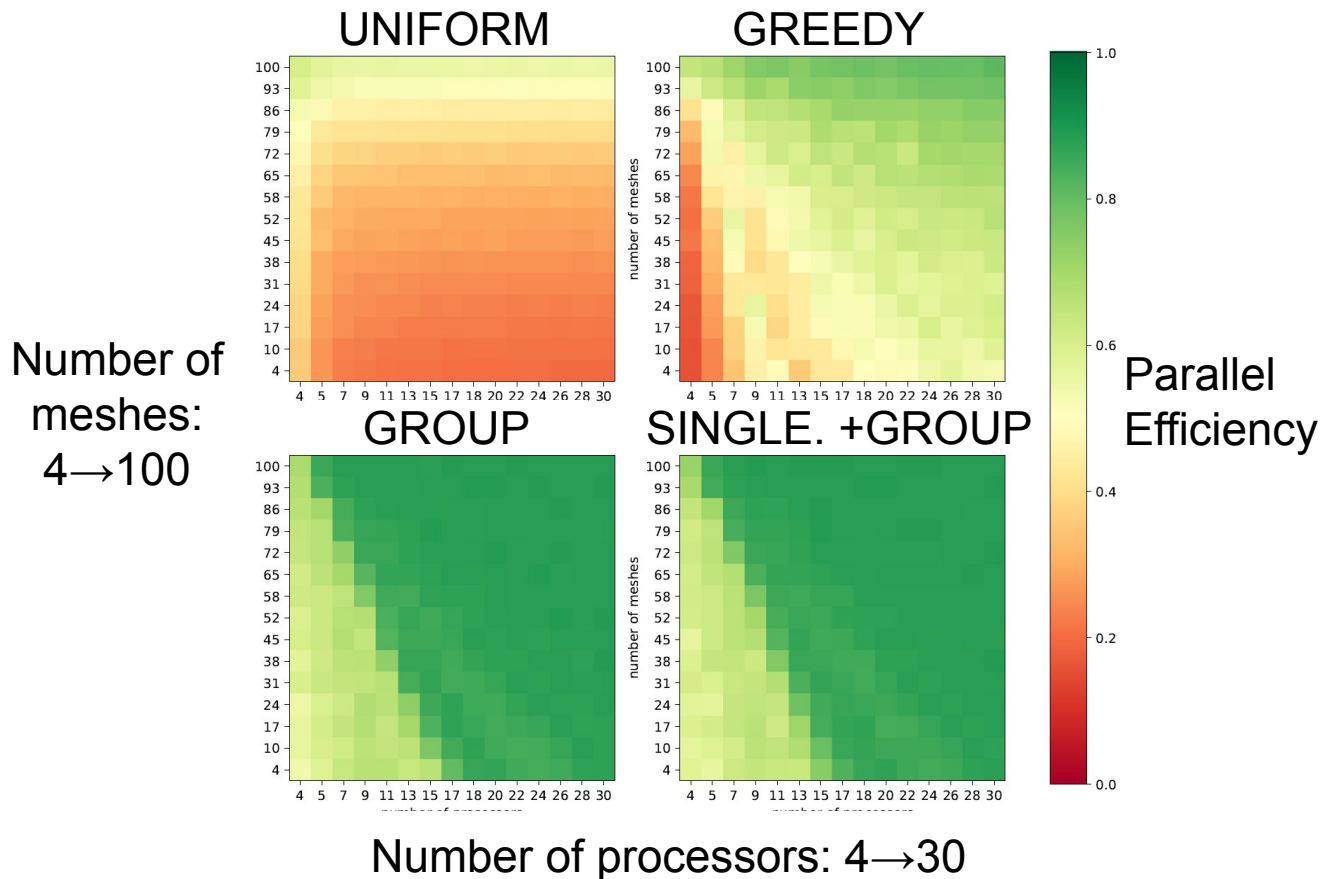
$$T_{total} = I \left[\frac{\frac{N}{P} + C_g \left(\frac{N}{P} \right)^{2/3} + n_{1/2}}{P_{peak}} + \beta \left(\frac{N}{P} \right)^{2/3} + \alpha \right]$$

$$T_{ideal} = \frac{I}{P_{peak}} \left(\frac{N}{P} + n_{1/2} \right)$$

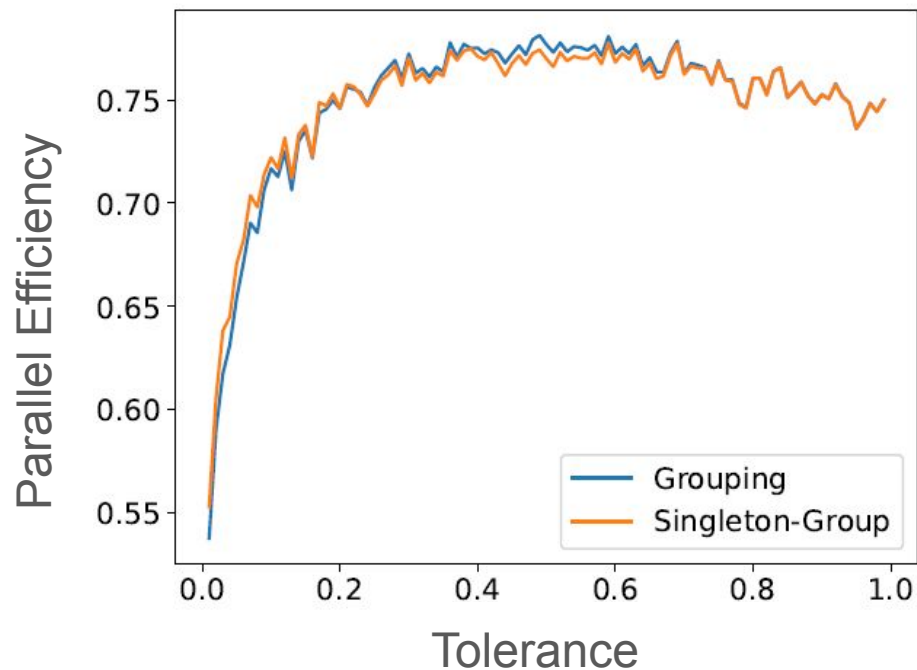
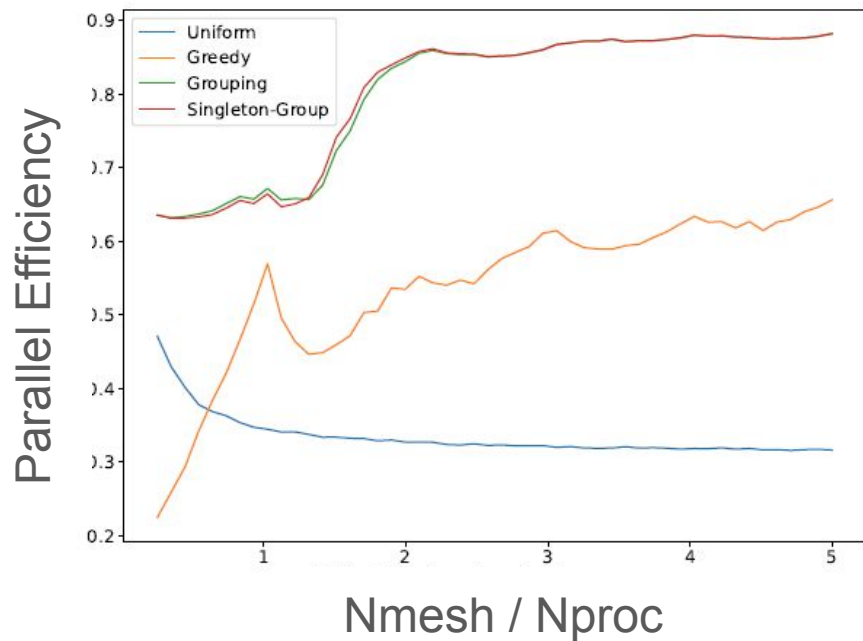
$$\epsilon = \frac{T_{ideal}}{T_{total}}$$

Parameter	Value
$P_{peak}(throughput)$	100M cells/second
Bandwidth	200 Gb/s
$n_{1/2}$	2 million cells
n_{fields}	5
β (comm factor)	$n_{fields} \times 8 / \text{bandwidth}$
C_g (number of ghost layers)	1
α (latency)	5×10^{-6} seconds
N (total grid size)	varies
P (Number of GPUs)	varies
I (iterations)	varies

Parallel efficiency comparison



Parallel efficiency sensitivity



Medium (8 PEs)	M_1 : (129,129,193)	3.21	43.32	$L = \begin{bmatrix} 5 & 2 & 9 & & \\ 5 & 2 & 8 & & \\ 5 & 2 & & & \\ 5 & 2 & 7 & & \\ 5 & 2 & & & \\ 1 & 4 & 3 & 6 & \\ 1 & 4 & 3 & & \\ 1 & 4 & 3 & & \end{bmatrix}$	$\begin{bmatrix} 5.80 \\ 5.80 \\ 5.10 \\ 5.28 \\ 5.10 \\ 5.53 \\ 5.34 \\ 5.34 \end{bmatrix}$	6.67%
	M_2 : (513,129,193)	12.77				
	M_3 : (129,257,193)	6.40				
	M_4 : (513,65,193)	6.44				
	M_5 : (513,129,193)	12.77				
	M_6 : (129,7,193)	0.17				
	M_7 : (129,7,193)	0.17				
	M_8 : (513,7,193)	0.69				
	M_9 : (513,7,193)	0.69				
Fine (12 PEs)	M_1 : (193,193,289)	10.76	143.47	$L = \begin{bmatrix} 1 & 4 & 9 & 5 & 2 & & \\ 1 & 4 & 9 & 5 & 2 & 7 & \\ 1 & 4 & 9 & 5 & 2 & & \\ 1 & 4 & 9 & 5 & 2 & & \\ 1 & 4 & 9 & 5 & 2 & 6 & \\ 1 & 4 & 9 & 5 & 2 & & \\ 1 & 4 & 9 & 5 & 2 & & \\ 1 & 4 & 9 & 5 & 2 & & \\ 1 & 4 & 9 & 5 & 2 & & \\ 1 & 4 & 9 & 5 & 2 & & \\ 8 & 3 & & & & & \\ 8 & 3 & & & & & \end{bmatrix}$	$\begin{bmatrix} 11.97 \\ 12.36 \\ 11.97 \\ 11.97 \\ 12.36 \\ 11.97 \\ 11.97 \\ 11.97 \\ 11.97 \\ 11.97 \\ 11.51 \\ 11.51 \end{bmatrix}$	3.24 %
	M_2 : (769,193,289)	42.89				
	M_3 : (193,385,289)	21.47				
	M_4 : (769,97,289)	21.56				
	M_5 : (769,193,289)	42.89				
	M_6 : (193,7,289)	0.39				
	M_7 : (193,7,289)	0.39				
	M_8 : (769,7,289)	1.56				
	M_9 : (769,7,289)	1.56				

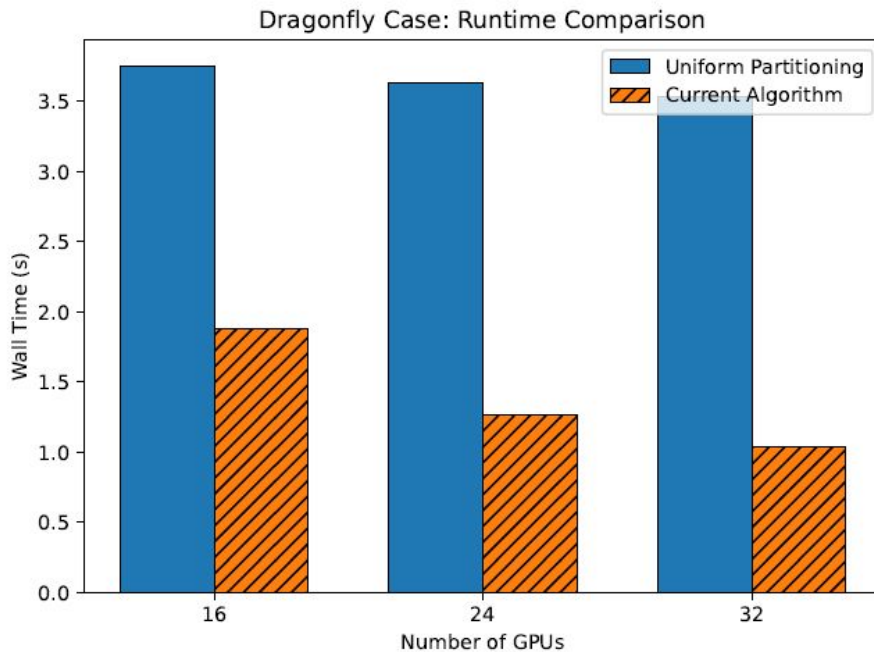
Algorithm dissemination

- Code published in github with MIT license
 - <https://github.com/lsitaraman677/Optimal-mesh-distribution>
- Can be used as a pre-processor for any CFD code that uses multiple meshes to determine the partitioning distribution
- Implemented in CFD code HPCMP CREATE-AV Helios by the code developers
- Application tests performed for two large test cases
 - Dragonfly titan probe (25 meshes)
 - UH-60A helicopter (9 meshes)
 - XV-15 tilt-rotor aircraft (9 meshes)

Dragonfly evaluation with Helios



(b) Dragonfly https://en.wikipedia.org/wiki/Dragonfly_Titan_space_probe



(b) Dragonfly wall clock time per time step

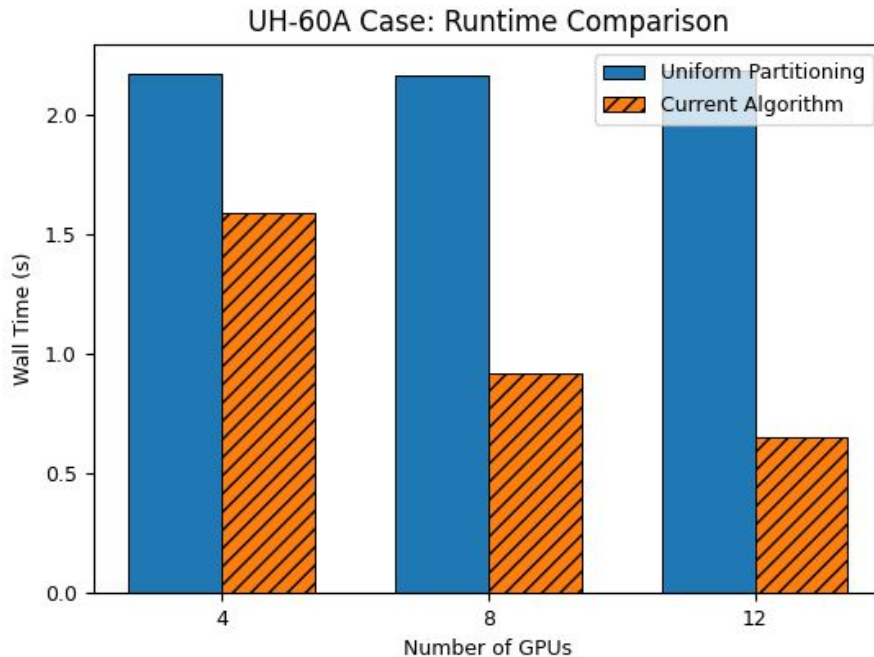
- 3x improvement in simulation speed on 32 GPUs
- Improved scalability of execution

UH-60A evaluation with Helios

9 meshes :
Fuselage + 8 blades



(c) UH-60A <https://ntrs.nasa.gov/api/citations/20110011257/downloads/20110011257.pdf>



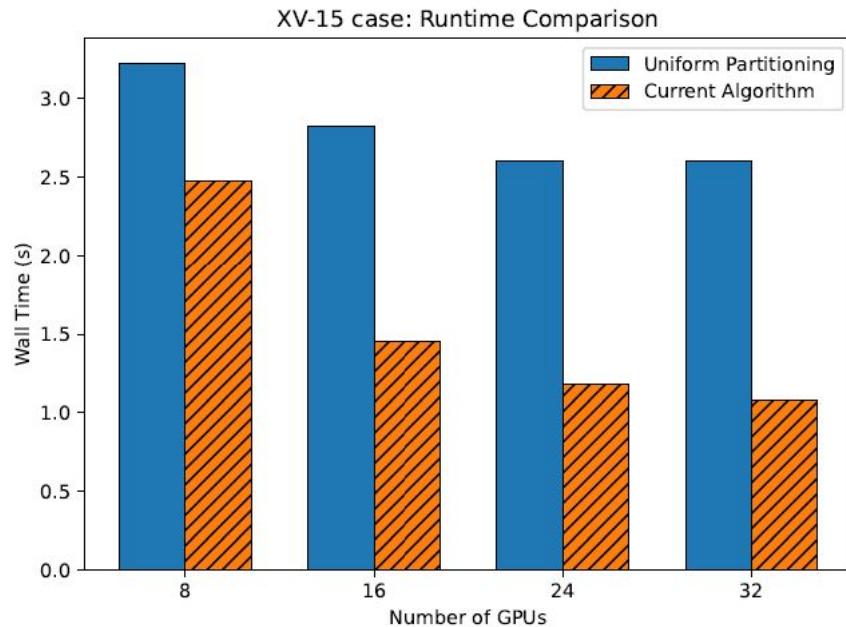
(c) UH-60A wall clock time per time step

- 3.5x improvement in simulation speed on 32 GPUs
- With uniform partitioning, increased GPUs does not decrease wall time

XV-15 evaluation with Helios



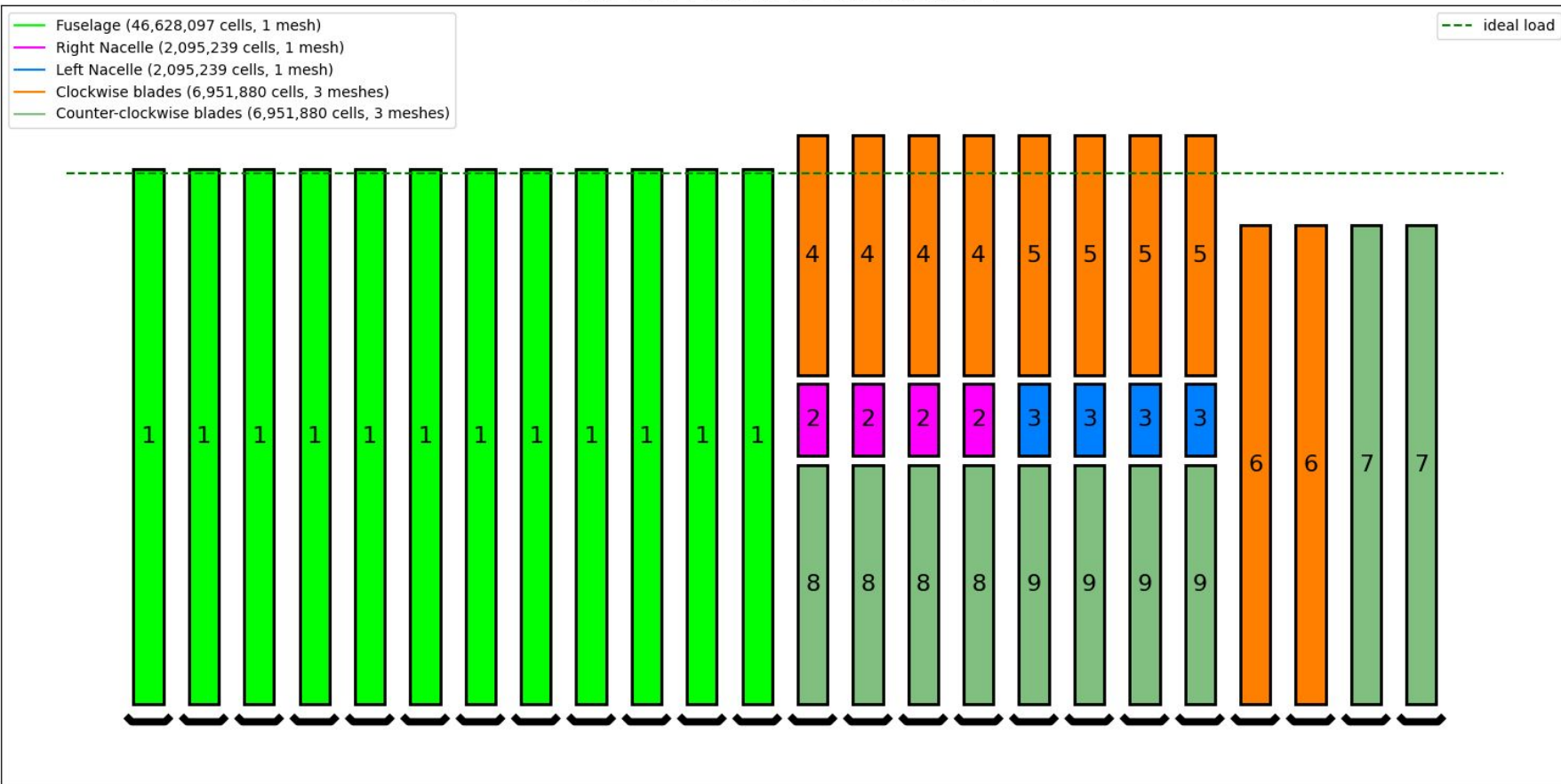
(a) XV-15 aircraft https://en.wikipedia.org/wiki/Bell_XV-15



(a) XV-15 wall clock time per time step

- Current partitioning algorithm shows 2.5x improvement over uniform partitioning
- Unsteady Computations require several thousand timesteps - large overall time savings

XV-15 - Example Distribution with 24 GPUs



Conclusions

- Developed a heuristic algorithm for distributing multiple meshes on comparable number of GPUs
- Canonical evaluation showed much improved parallel efficiency compared to uniform and greedy partitioning schemes
- Code published open source on github and was evaluated within CREATE-AV Helios
 - 2.5-3.5x speedup for large test cases
 - Improved scalability compared to uniform partitioning

Acknowledgements

The author acknowledges Dr. Shirzad Hosseinverdi of the HPCMP CREATE-AV program for incorporating the developed algorithm into the RAPIDUS¹ module within the CREATE-AV Helios code base and providing simulation time comparisons with the uniform partitioning algorithm.

¹Premaratne, P. D., Hosseinverdi, S., and Jude, D., "Verification and Validation of a Performance-Portable Unstructured Navier-Stokes Solver (RAPIDUS)," AIAA SCITECH 2025 Forum, 2025, p. 0892.